The Reality Behind PKI Revocation Checking – Michael Waterman

michaelwaterman.nl/2026/05/25/the-reality-behind-pki-revocation-checking

Michael Waterman May 25, 2026

Last week I attended an interesting PKI training from CQURE. I never really had any formal PKI training before, mostly because I've spent years learning it the way many infrastructure engineers do, by breaking things in labs, fixing production issues, and occasionally questioning my life choices while staring at certutil output at 2 AM.

Still, I thought it would be fun to join. Most of the material was already familiar, but I met interesting people, had some good discussions, and definitely learned a few new things along the way. If you want to get into Microsoft PKI, I can genuinely recommend the training. PKI is one of those subjects that somehow manages to be both incredibly boring and extremely fascinating at the same time.

One of the topics we discussed was revocation checking. In the Microsoft world, this usually means Certificate Revocation Lists (CRLs) or the Online Certificate Status Protocol (OCSP). What many people misunderstand, however, is that revocation checking is not some universally enforced security mechanism. Whether revocation is actually checked often depends entirely on the application, service, operating system, or even the exact API being used underneath.

Take web browsers for example. Most modern browsers do not aggressively hard-fail on revocation checking anymore. In many cases they either soft-fail, rely on cached information, or use mechanisms like CRLSets and CRLite (looking at you Firefox) instead of traditional CRL retrieval. Has anyone actually seen a real "*This certificate has been revoked*" message in recent years? I honestly can't remember the last time I did.

And that operational reality becomes even more interesting once you start looking at Microsoft Active Directory Certificate Services itself. Because ironically, one of the systems that can become surprisingly strict about CRL availability… is the CA server itself.

Revocation checking

When a certificate is presented during something like a TLS connection, smart card authentication, or code signing validation, the system validating the certificate usually performs more checks than just verifying the signature chain. One of those checks can be revocation checking.

In simple terms, revocation checking answers a fairly important question: *"Has this certificate been explicitly invalidated before its expiration date?"*

That can happen for several reasons. Maybe the private key was compromised, maybe the server was decommissioned, or maybe an administrator somewhere accidentally issued a certificate to the wrong system at 4 PM on a Friday afternoon. These things happen more often than people like to admit. Traditionally, this validation is performed through a Certificate Revocation List (CRL). A CRL is essentially a signed list published by the Certificate Authority containing serial numbers of certificates that should no longer be trusted. Clients retrieve this list through the CRL Distribution Point (CDP) embedded inside the certificate itself. On Windows systems, revocation checking behavior can actually be observed fairly easily with certutil. For example, the following command shows the locally cached CRLs on a system:

```
PS C:\Users\administrator.CORP> certutil -urlcache crl

http://ctldl.windowsupdate.com/msdownload/update/v3/static/trustedr/en/authrootstl.cab
http://pki.trust.corp.michaelwaterman.nl/crl/Corp-Root-CA.crl
WinHttp Cache entries: 2
```

Example output of my issuing CA

That cache turns out to be an important detail. Because in many enterprise environments, revocation checking is not necessarily happening "live" every single time a certificate is validated. Quite often, Windows simply relies on previously cached CRLs that may still be considered valid according to their lifetime settings. And that operational nuance is where things start becoming interesting. Because once you realize revocation checking is heavily dependent on caching behavior, timeout handling, application logic, and soft-fail scenarios, you also start realizing that "the CRL server is offline" does not automatically mean "everything immediately stops working."

In fact, in many environments, administrators may not even notice a CRL distribution point is unavailable for quite some time. Somewhere out there, there is probably still a production server happily using a CRL that expired weeks ago, and nobody knows because "the application still works." That sentence probably raised the blood pressure of a few people.

What happens when revocation checking fails?

One of the biggest misconceptions around PKI is the idea that certificate validation immediately breaks the moment a CRL becomes unavailable. In reality, things are usually far more complicated than that.

Some applications perform strict revocation checking and hard-fail when the CRL or OCSP responder cannot be reached. Others simply continue operating and treat the revocation status as "unknown." Some rely entirely on cached CRLs. Others barely check revocation at all. And then there are applications that behave differently depending on operating system version, timeout behavior, or underlying CryptoAPI settings. Because of course they do.

This creates an interesting operational reality inside enterprise environments. A CRL distribution point may already be offline for hours, days, or sometimes even weeks without anybody noticing. Existing TLS sessions

continue working, browsers remain happy, internal applications keep functioning, and monitoring systems stay suspiciously quiet.

Until suddenly one specific system starts failing, usually at the worst possible moment.

Smart card logons may stop working (they actually immediately do when they can't check the crl). An NPS server performing EAP-TLS validation suddenly refuses authentication requests. A VPN solution starts rejecting certificates. Or an administrator discovers that a certificate authority itself refuses to start because it cannot successfully perform revocation checking on its own certificate chain, and that last scenario is where things become particularly interesting.

Why your browser doesn't care

Years ago, browsers used to be significantly more aggressive with revocation checking. In theory, this idea sounded perfectly reasonable, if a certificate was compromised or revoked, clients should immediately stop trusting it. Simple. Unfortunately, the real world had other plans.

Traditional CRL checking introduced several practical problems almost immediately. Browsers suddenly became dependent on external CRL distribution points that were sometimes slow, unreachable, misconfigured, or simply forgotten. OCSP improved this somewhat, but introduced its own challenges, additional latency (like10% doesn't even answer), privacy concerns (the responder knows the site you're visiting), responder outages, and yet another infrastructure dependency that somebody somewhere had to maintain.

And like many things in enterprise infrastructure, revocation checking turned out to be far less reliable than the PowerPoint diagrams suggested. At some point browser vendors realized something uncomfortable, strict revocation enforcement could easily break large parts of the internet because of temporary availability issues that had nothing to do with actual certificate compromise. A CRL server outage should probably not prevent millions of users from opening websites. That operational reality slowly pushed browsers toward soft-fail behavior. Modern browsers all handle revocation slightly differently:

Browser	Revocation Behavior
Google Chrome	Primarily relies on CRLSets and soft-fail behavior
Microsoft Edge	Similar behavior through Chromium
Mozilla Firefox	Uses OneCRL and OCSP * soft-fail by default
Safari	Relies heavily on OCSP and Apple trust infrastructure

* Firefox used OCSP up until version 142, see: CRLite: Fast, private, and comprehensive certificate revocation checking in Firefox – Mozilla Hacks – the Web developer blog

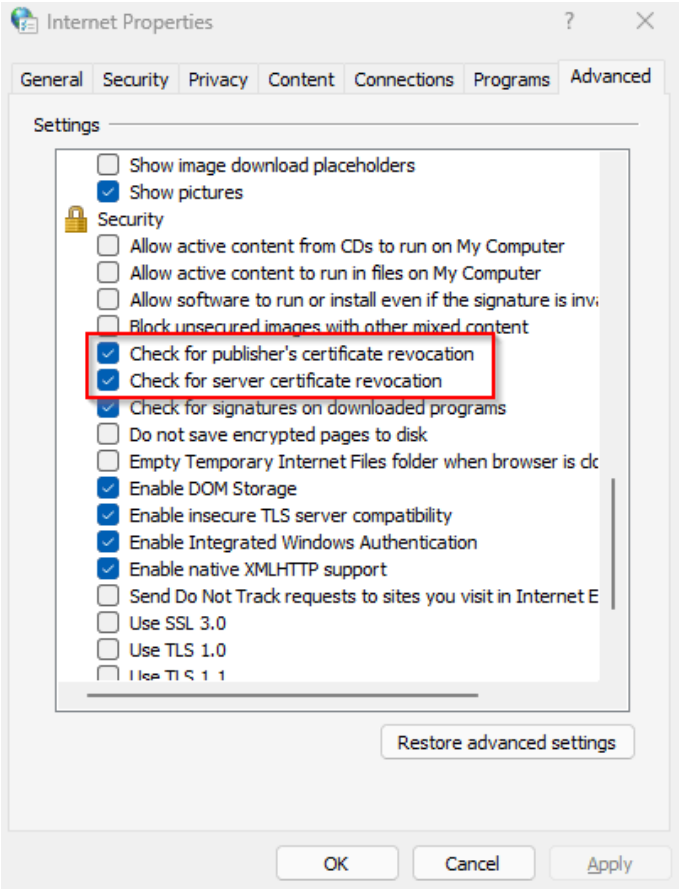
In practice, this means most browsers no longer perform aggressive "live" CRL validation for every TLS connection. Instead, they rely on cached information, vendor-managed revocation lists, or selective blocking mechanisms for known compromised certificates.

This also explains why the classic "*This certificate has been revoked*" browser warning has become surprisingly rare. In many situations, browsers prioritize availability and user experience over strict revocation enforcement. Whether that is the correct security decision is an entirely different discussion. Windows itself still exposes some of this behavior through legacy certificate validation settings. These can be found in Internet Options under:

Internet Options → Advanced

The most relevant options are:

- Check for publisher's certificate revocation
- Check for server certificate revocation



Windows revocation settings enabled by default.

Behind the scenes, these settings influence several WinINet and CryptoAPI dependent applications. That last part is important, because many administrators assume these settings only affect Microsoft Edge. They do not. Quite a lot of Windows software still depends on the underlying Windows certificate validation APIs. The settings themselves are stored in the following registry location:

```
HKCU\Software\Microsoft\Windows\CurrentVersion\WinTrust\Trust Providers\Software Publishing
```

The `State` value controls several trust validation behaviors, including revocation checking.

Setting	Effect
Check for publisher's certificate revocation	Verifies code signing certificate revocation
Check for server certificate revocation	Verifies TLS/SSL certificate revocation

You can also enable server certificate revocation checking through PowerShell:

```
Set-ItemProperty `
 -Path "HKCU:\Software\Microsoft\Windows\CurrentVersion\Internet Settings" `
 -Name CertificateRevocation `
 -Value 1
```

Enterprise browsers

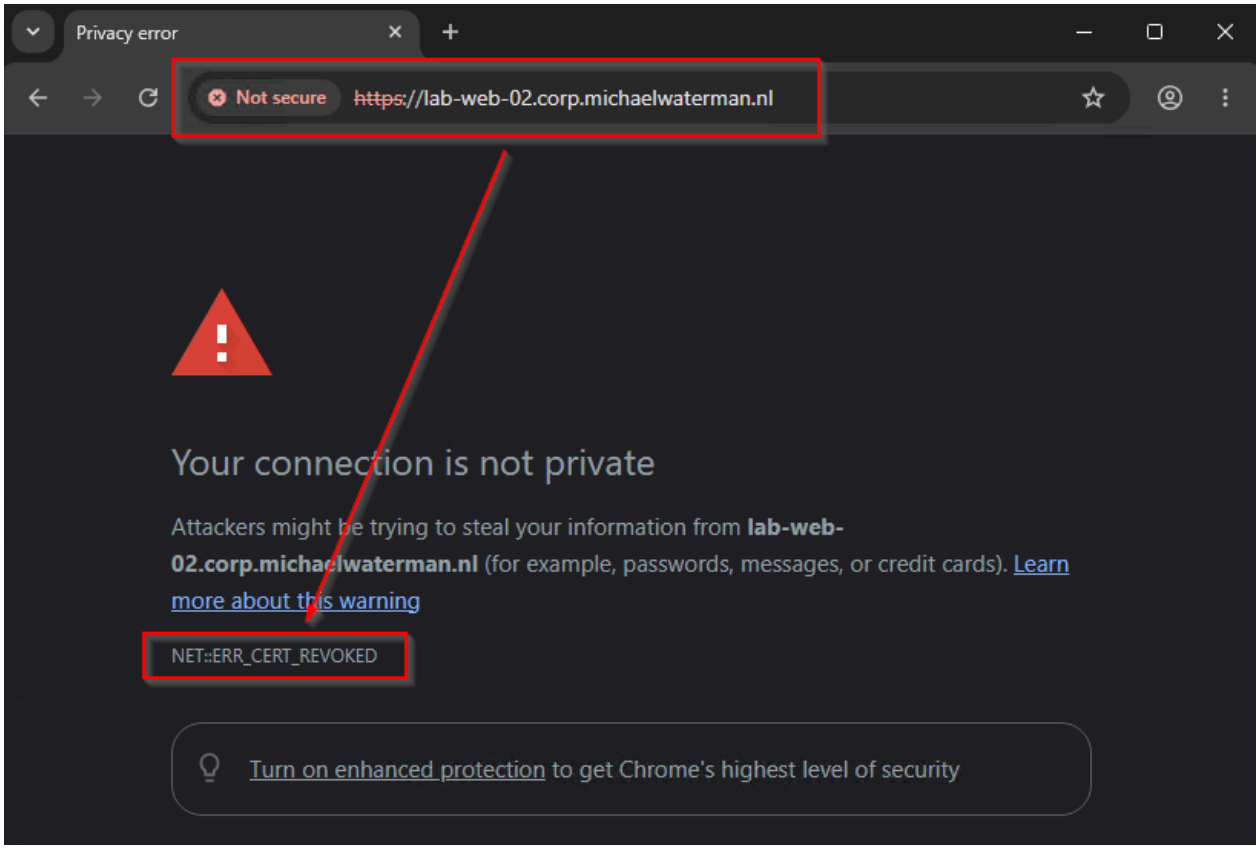
For enterprise environments, both Google Chrome and Microsoft Edge provide administrative controls to re-enable traditional online revocation checking. Interestingly, Microsoft and Google even distinguish between general internet revocation checking and stricter validation specifically for internally trusted certification authorities. For general revocation checking with soft-fail behavior, the following policies can be configured:

Browser	Registry Path	Policy
Chrome (Machine)	`HKLM\Software\Policies\Google\Chrome`	`EnableOnlineRevocationChecks=1`
Chrome (User)	`HKCU\Software\Policies\Google\Chrome`	`EnableOnlineRevocationChecks=1`
Edge (Machine)	`HKLM\Software\Policies\Microsoft\Edge`	`EnableOnlineRevocationChecks=1`
Edge (User)	`HKCU\Software\Policies\Microsoft\Edge`	`EnableOnlineRevocationChecks=1`

This enables online CRL and OCSP checking, but still allows the browser to continue if the revocation information cannot be retrieved. In other words: soft-fail behavior. More interesting however is the second policy, which specifically targets certificates chaining to locally trusted enterprise certification authorities:

Browser	Registry Path	Policy
Chrome (Machine)	`HKLM\Software\Policies\Google\Chrome`	`RequireOnlineRevocationChecksForLocalAnchors=1`
Chrome (User)	`HKCU\Software\Policies\Google\Chrome`	`RequireOnlineRevocationChecksForLocalAnchors=1`
Edge (Machine)	`HKLM\Software\Policies\Microsoft\Edge`	`RequireOnlineRevocationChecksForLocalAnchors=1`
Edge (User)	`HKCU\Software\Policies\Microsoft\Edge`	`RequireOnlineRevocationChecksForLocalAnchors=1`

This changes the behavior significantly. If revocation information for internally trusted certificates cannot be retrieved, Chromium-based browsers will hard-fail the connection and treat the certificate as invalid. Architecturally, this actually makes a lot of sense. Public internet PKI is messy, distributed, and partially outside your control. Internal enterprise PKI environments however are supposed to have predictable CRL availability, managed infrastructure, and controlled network paths. At least in theory.



Chrome detects a certificate that's revoked and issues a clear warning.

Whether enabling strict revocation checking for internal certificates is a good idea depends heavily on the operational maturity of the PKI environment itself. In a properly maintained enterprise PKI with highly available CRL distribution points, monitoring, and solid lifecycle management, enabling stricter revocation enforcement can absolutely improve security.

In less mature environments, however, these settings can become surprisingly dangerous. Suddenly CRL publication intervals, IIS virtual directories, proxy behavior, firewall rules, offline subordinate CAs, and forgotten legacy CDP locations become production-critical dependencies. Somewhere out there, there is still an internal application depending on a CRL endpoint hosted on a Windows Server 2008 machine nobody dares to reboot.

Firefox behaves somewhat differently here. Mozilla relies more heavily on OCSP (Up until version 142), OneCRL, and CRLite mechanisms instead of the Chromium revocation model. During testing, I actually struggled to get Firefox to reject a revoked certificate from an internal Microsoft PKI environment, even after explicitly enabling stricter OCSP behavior and disabling CRLite. Most likely this was caused by Firefox simply handling enterprise revocation scenarios differently than expected, I could not find any additional info on the subject. Ironically, that experience perfectly illustrates the larger problem with certificate revocation on modern systems, what sounds straightforward on paper quickly becomes surprisingly nuanced once browsers, caching, timeout handling, and real-world operational behavior enter the picture.

What to do?

Now the interesting question becomes, should you actually enable strict revocation checking? Architecturally speaking, the answer is usually "yes." If a certificate is compromised, clients should ideally reject it as quickly as possible. That is the entire point of revocation infrastructure. Operationally however, things become more complicated.

Strict revocation enforcement introduces dependencies on CRL availability, OCSP responsiveness, proxy configurations, firewall rules, caching behavior, and timeout handling. Somewhere inside almost every enterprise environment there is still a forgotten application server that suddenly stops functioning because it cannot retrieve a CRL from a URL that nobody has looked at in six years. That is why many modern platforms moved toward soft-fail behavior. Availability almost always wins from strict security enforcement once production outages enter the conversation.

Personally, I would still recommend *enabling revocation checking in enterprise environments*, especially for internal PKI deployments, privileged authentication scenarios, smart card environments, and code signing validation. But administrators should understand an important reality before doing so:

> ***the moment revocation checking becomes mandatory, CRL infrastructure suddenly becomes production-critical infrastructure.***

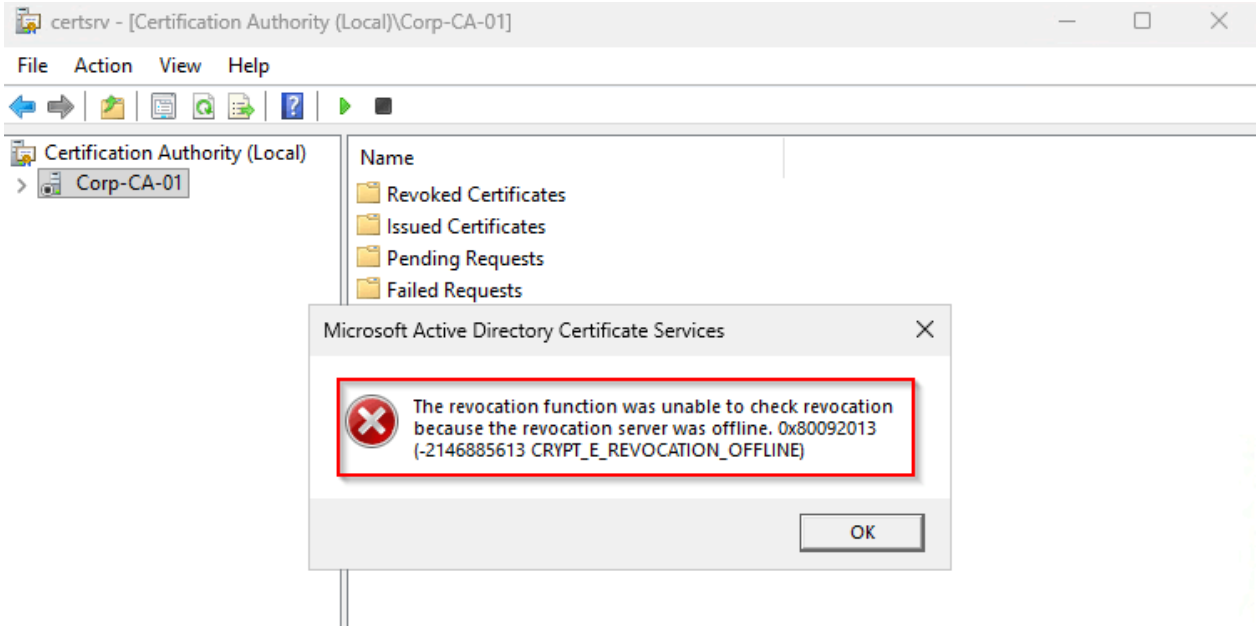
And that realization brings us directly to Active Directory Certificate Services itself. Because ironically, one of the systems that can become surprisingly strict about CRL availability… is the CA server itself.

When the CA itself refuses to start

Up until now, most of the revocation behavior we discussed was relatively forgiving. Browsers soft-fail, applications rely on caching, and many systems continue functioning perfectly fine even when CRL distribution points temporarily disappear. Active Directory Certificate Services, however, can be a very different story.

One of the more frustrating scenarios in Microsoft PKI is a CA service refusing to start because revocation checking on the Root CA CRL fails. And yes, that sounds slightly absurd the first time you encounter it. The error usually looks something like this:

The revocation function was unable to check revocation because the revocation server was offline. 0x80092013 (-2146885613 CRYPT_E_REVOCATION_OFFLINE)



The CA could not check the revocation list

Ironically, the CA that publishes revocation information can sometimes become dependent on revocation checking itself, although in this case it's the CRL of the parent CA that's causing the issue. Somewhere in the certificate chain validation process, Windows attempts to validate the CA certificate itself. As part of that process, it will try to retrieve revocation information from the CRL published by the parent or Root CA through the configured CRL Distribution Points (CDPs). If that retrieval fails in certain situations, the Certificate

Services service may simply refuse to start. And this is where the operational reality of PKI becomes very visible.

Technically speaking, revocation checking is absolutely correct here. If the certificate chain cannot be validated properly, trust should not be established. From a pure security perspective, the logic is completely sound. Operationally however, it can become extremely frustrating. From what I have seen, the underlying issue is often caused by unreachable CDP locations. Even if the issuing CA itself is perfectly functional, Windows may still attempt revocation retrieval against HTTP or LDAP locations that are no longer reachable. You can often observe this behavior manually with:

```
certutil -urlfetch -verify <CAcertificate.cer>
```

```
-------- CERT_CHAIN_CONTEXT --------
ChainContext.dwInfoStatus = CERT_TRUST_HAS_PREFERRED_ISSUER (0x100)
ChainContext.dwErrorStatus = CERT_TRUST_REVOCATION_STATUS_UNKNOWN (0x40)
ChainContext.dwErrorStatus = CERT_TRUST_IS_OFFLINE_REVOCATION (0x1000000)

SimpleChain.dwInfoStatus = CERT_TRUST_HAS_PREFERRED_ISSUER (0x100)
SimpleChain.dwErrorStatus = CERT_TRUST_REVOCATION_STATUS_UNKNOWN (0x40)
SimpleChain.dwErrorStatus = CERT_TRUST_IS_OFFLINE_REVOCATION (0x1000000)

CertContext[0][0]: dwInfoStatus=102 dwErrorStatus=1000040
  Issuer: CN=Corp-Root-CA
  NotBefore: 24-5-2026 17:04
  NotAfter: 24-5-2031 17:14
  Subject: CN=Corp-CA-01, DC=corp, DC=michaelwaterman, DC=nl
  Serial: 560000000316c2cc3b462d61a8000000000003
  Template: SubCA
  Cert: ce3ca19c4dc50cdcf8eff58ec4ed591f85a71464
  Element.dwInfoStatus = CERT_TRUST_HAS_KEY_MATCH_ISSUER (0x2)
  Element.dwInfoStatus = CERT_TRUST_HAS_PREFERRED_ISSUER (0x100)
  Element.dwErrorStatus = CERT_TRUST_REVOCATION_STATUS_UNKNOWN (0x40)
  Element.dwErrorStatus = CERT_TRUST_IS_OFFLINE_REVOCATION (0x1000000)
  ----------------  Certificate AIA  ----------------
  Failed "AIA" Time: 0 (null)
    Error retrieving URL: The operation timed out 0x80072ee2 (WinHttp: 12002 ERROR_WINHTTP_TIMEOUT)
    http://pki.trust.corp.michaelwaterman.nl/aia/Corp-Root-CA.crt

  ----------------  Certificate CDP  ----------------
  Failed "CDP" Time: 0 (null)
    Error retrieving URL: The operation timed out 0x80072ee2 (WinHttp: 12002 ERROR_WINHTTP_TIMEOUT)
    http://pki.trust.corp.michaelwaterman.nl/crl/Corp-Root-CA.crl
```

Error messages during verification

And that is the interesting contradiction at the heart of Microsoft PKI revocation behavior. Modern browsers became increasingly tolerant because strict revocation checking broke availability too often. Active Directory Certificate Services however can still become surprisingly strict under certain circumstances. Fortunately, Microsoft included an escape hatch for exactly these situations. Although it is one of those registry flags you preferably never need to touch unless you fully understand the implications.

Important disclaimer

It is important to stress that the procedure described below should generally be considered a temporary break-glass recovery procedure rather than the preferred way to resolve revocation issues.

In properly maintained PKI environments, the correct solution is usually to restore revocation availability itself by publishing fresh CRLs from the parent CA and restoring access to the configured CDP locations. In some scenarios, administrators may also temporarily import a valid parent CRL into the local certificate store using `certutil -addstore`, allowing normal chain validation to continue without disabling revocation checking entirely.

The described `CRLF_REVCHECK_IGNORE_OFFLINE` flag is primarily useful during abnormal recovery situations where restoring CRL availability is temporarily impossible, impractical, or time-sensitive. Architecturally speaking, highly available CDP locations, proper CRL lifecycle management, and documented recovery procedures should always remain the preferred long-term solution.

The emergency registry flag

Fortunately, Microsoft included an escape hatch for situations where a CA refuses to start because revocation checking fails. The following command tells the CA service to ignore offline revocation failures during startup:

```
certutil -setreg CA\CRLFlags +CRLF_REVCHECK_IGNORE_OFFLINE
```

```
C:\Users\administrator.CORP>certutil -setreg CA\CRLFlags +CRLF_REVCHECK_IGNORE_OFFLINE
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\CertSvc\Configuration\Corp-CA-01\CRLFlags:

Old Value:
  CRLFlags REG_DWORD = 2
    CRLF_DELETE_EXPIRED_CRLS -- 2

New Value:
  CRLFlags REG_DWORD = a (10)
    CRLF_DELETE_EXPIRED_CRLS -- 2
    CRLF_REVCHECK_IGNORE_OFFLINE -- 8
CertUtil: -setreg command completed successfully.
The CertSvc service may need to be restarted for changes to take effect.
```

Emergency CRL check bypass

Afterwards, restart the Certificate Services service, or reboot the server if you prefer solving enterprise problems the traditional way. What this setting essentially does is allow the CA to continue operating even when CRL retrieval fails because the revocation server is unreachable. This is useful during CDP location maintenance or during an incident. That said, this flag should ideally remain a temporary workaround rather than a permanent solution. If a production issuing CA continuously depends on this setting, there is usually a deeper issue hiding somewhere in the PKI design. Often this turns out to be an old CRL distribution point, a forgotten LDAP path, or some legacy IIS server nobody realized was still hosting revocation data.

Once the underlying CRL issue has been resolved, the setting can be reverted with the following command:

```
certutil -setreg CA\CRLFlags -CRLF_REVCHECK_IGNORE_OFFLINE
```

After restarting the Certificate Services service, the CA will again perform normal revocation checking behavior during startup. And ideally, that is where you want to end up eventually. The registry flag is useful during recovery or troubleshooting scenarios, but probably not something you want lingering around unnoticed for the next five years.

Troubleshooting revocation caching

One of the more confusing aspects of revocation checking on Windows is caching behavior. Administrators often revoke a certificate, publish a new CRL, test the connection again… and everything still works perfectly fine. Usually this is the point where people start questioning whether revocation checking is even happening at all, like we saw in the chapter on Browsers. In reality, Windows aggressively caches CRLs and OCSP responses locally. That behavior improves performance and availability, but it can make troubleshooting surprisingly frustrating.

The order in which Windows performs revocation checking is particularly important to understand:

1. First, Windows checks whether a valid CRL already exists in the local memory cache.

2. If not found there, it checks the local disk cache.
    - User: *C:\Users\<username>\AppData\LocalLow\Microsoft\ CryptnetUrlCache*
    - System: *C:\Windows\System32\config\systemprofile\AppData\LocalLow\ Microsoft\CryptnetUrlCache*
3. Third it will check anything available in the certificate store.
4. Only if no valid cached CRL is available does Windows attempt to retrieve a fresh CRL from the configured distribution point.

That explains why revocation testing often behaves differently than you would expect. Even after publishing a new CRL, systems may continue using older cached revocation data until the cache expires or is manually cleared. The first step during troubleshooting is usually checking what is currently cached:

Or for OCSP responses:

If you want to remove the local disk cache completely, use:

```
certutil -urlcache * delete
```

However, that still does not clear everything. Applications and services may continue using in-memory revocation data that remains loaded inside the running process itself. Browsers, IIS, LSASS, and various CryptoAPI consumers all behave slightly differently here. Sometimes restarting the application is enough. Sometimes the server itself becomes the troubleshooting step. Windows also provides a more aggressive option by invalidating all cached chain data immediately:

```
certutil -setreg chain\ChainCacheResyncFiletime @now
```

This effectively forces Windows to stop trusting all currently cached revocation data and retrieve fresh information again. If you want to disable it for 8 hours, you can use this little trick:

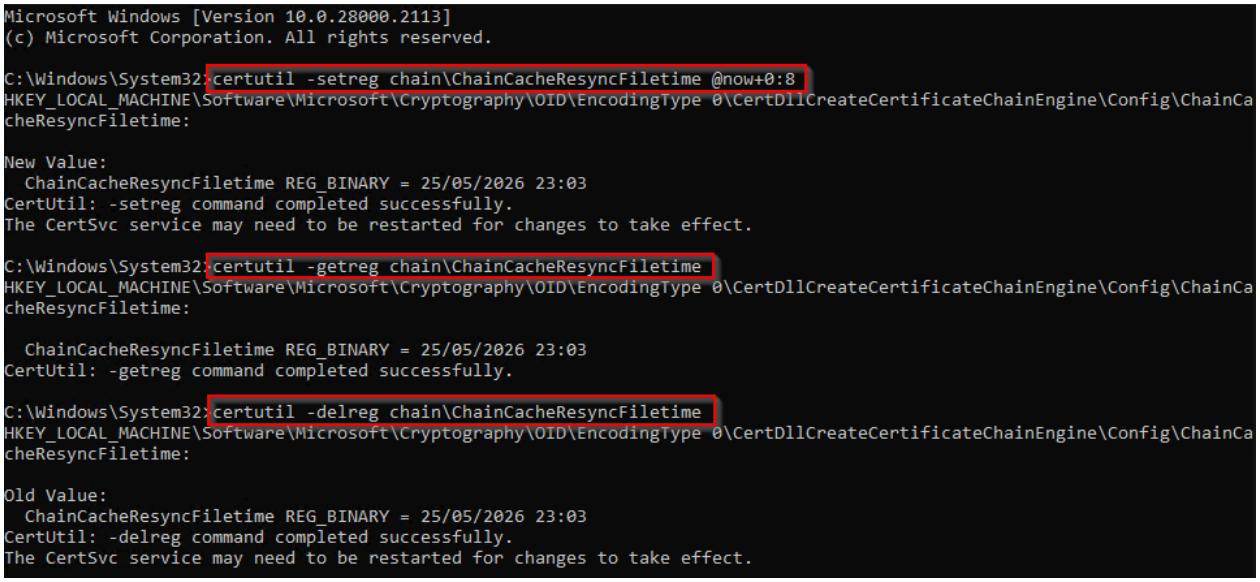
```
certutil -setreg chain\ChainCacheResyncFiletime @now+0:8
```

Viewing the current setting can be achieved with:

```
certutil -getreg chain\ChainCacheResyncFiletime
```

Once troubleshooting is complete, the setting can be removed with:

```
certutil -delreg chain\ChainCacheResyncFiletime
```



Modifying the cache values

Understanding this behavior becomes especially important during troubleshooting. Otherwise you may end up revoking certificates repeatedly while Windows happily continues trusting them because somewhere deep inside a cache layer, the old CRL is still considered valid.

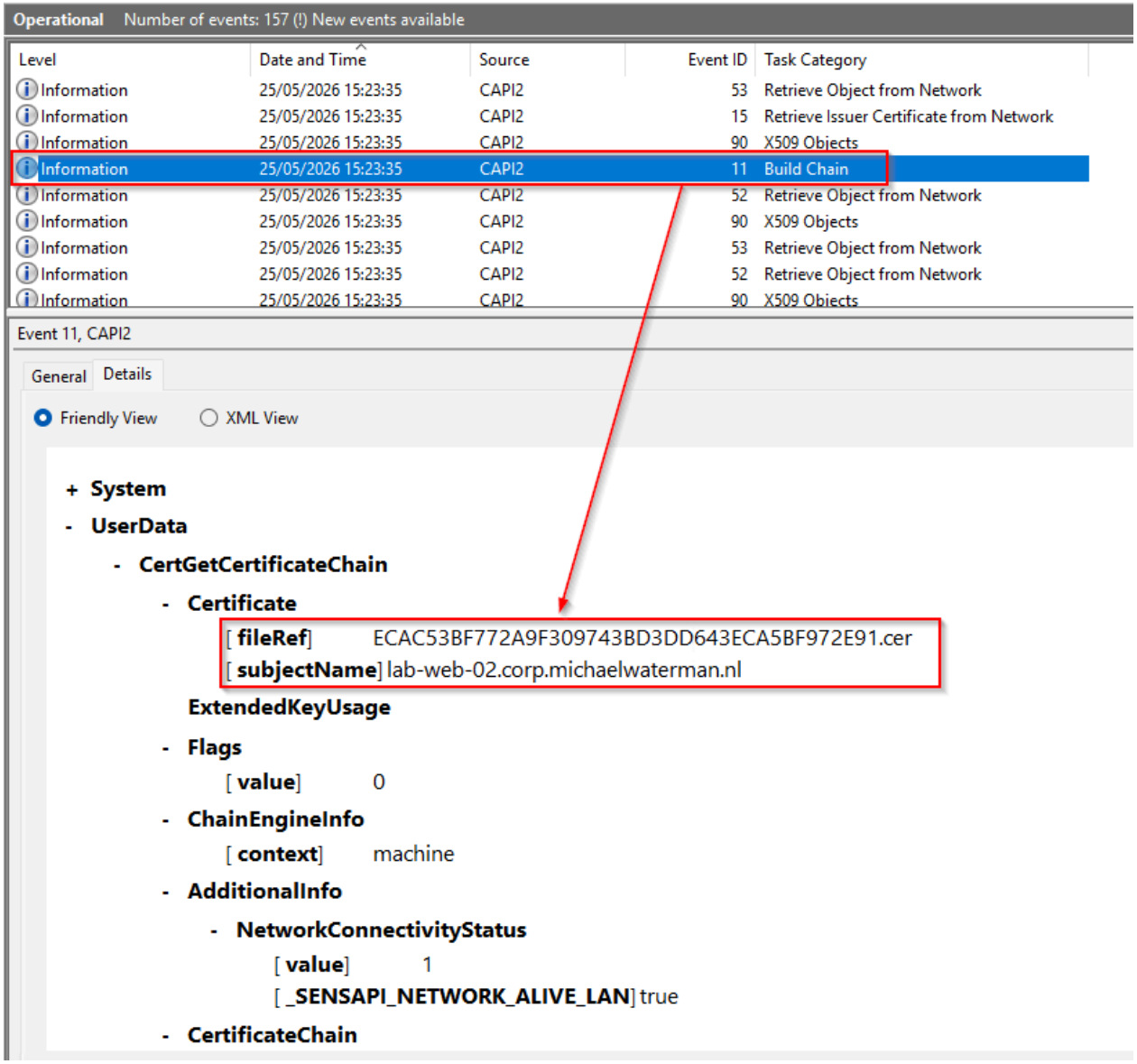
CAPI2 event log

For deeper troubleshooting, enabling the CAPI2 operational log is highly recommended. This is one of those logs every PKI person eventually discovers after spending far too long troubleshooting certificate issues blindly. The log can be enabled through:

Event Viewer → Applications and Services Logs →Microsoft → Windows → CAPI2 → Operational

Once enabled, Windows starts logging detailed certificate chain building, revocation retrieval, OCSP requests, CRL downloads, and trust validation activity. This becomes incredibly useful when trying to determine whether revocation checking is actually happening, using a cache or getting a timeout. Some particularly useful event IDs are:

Event ID	Description
11	Successful certificate chain building
30	Revocation checking details
41	Certificate chain validation failure
53	CRL retrieval operations
60	OCSP retrieval activity



CAPI2 Log

The CAPI2 log quickly reveals something many administrators eventually discover the hard way, revocation checking on Windows is rarely a simple “yes or no” process. It is usually a complicated mixture of cache states, timeout handling, application behavior, retrieval methods, and fallback logic that slowly turns into a troubleshooting rabbit hole of its own.

Final thoughts

Certificate revocation sounds deceptively simple on paper, revoke a certificate, publish a CRL, and clients stop trusting it. The reality turns out to be a lot messier in my experience. Modern browsers prioritize availability, Windows heavily relies on caching, applications all behave differently, and even Active Directory Certificate Services itself can occasionally refuse to function because revocation checking becomes too strict. Somewhere between security theory and operational reality, revocation checking slowly turned into a balancing act between trust, availability, and survivability (mine usually).

That probably describes enterprise PKI better than anything else ever could.

Hope you found this, longer than intended, post useful. As always, leave a comment, ask a question or reach out to me. Until next time!

References

https://www.gradenegger.eu/en/google-chrome-does-not-check-revocation-status-of-certificates

https://learn.microsoft.com/en-us/troubleshoot/windows-server/licensing-and-activation/error-when-you-validate-copy-windows

https://learn.microsoft.com/en-us/windows/win32/api/wintrust/nf-wintrust-winverifytrust

https://www.stigviewer.com/stigs/microsoft_dotnet_framework_40/2025-05-16/finding/V-225224